

Python questions

29 July 2024 19:57

You have the following marks for four students:

Student 1: 85

Student 2: 90

Student 3: 78

Student 4: 92

Write a Python script to:

Calculate the total marks of all students.

Calculate the average marks.

Find the highest and lowest marks.

Example Solution:

```
# Student marks
marks1 = 85
marks2 = 90
marks3 = 78
marks4 = 92

# Total marks
total_marks = marks1 + marks2 + marks3 + marks4

# Average marks
average_marks = total_marks / 4

# Highest and lowest marks
highest_marks = max(marks1, marks2, marks3, marks4)
lowest_marks = min(marks1, marks2, marks3, marks4)

# Print results
print("Total Marks:", total_marks)
print("Average Marks:", average_marks)
print("Highest Marks:", highest_marks)
print("Lowest Marks:", lowest_marks)
```

You have three students' marks: student1, student2, and student3. Write a Python code snippet to:

Calculate the sum of all three students' marks.

Calculate the average of the marks.

Determine the total number of students.

Calculate the percentage of marks for each student.

Assume the marks for the students are:

student1 = 85

student2 = 90

student3 = 78

Write your code to perform these calculations.

Solution

Here's the Python code to achieve the calculations:

```
# Marks of the students
student1 = 85
student2 = 90
student3 = 78

# Total marks
total_marks = student1 + student2 + student3

# Number of students
num_students = 3

# Average marks
average_marks = total_marks / num_students

# Percentage calculation
percentage_student1 = (student1 / 100) * 100
percentage_student2 = (student2 / 100) * 100
percentage_student3 = (student3 / 100) * 100

# Output the results
print("Total Marks:", total_marks)
print("Average Marks:", average_marks)
print("Percentage of Student 1:", percentage_student1, "%")
print("Percentage of Student 2:", percentage_student2, "%")
print("Percentage of Student 3:", percentage_student3, "%")
```

Explanation

Total Marks: Sum of all students' marks.

Average Marks: Total marks divided by the number of students.

Percentage: Each student's marks as a percentage of the maximum possible marks (which is 100 in this case).

Practice Question

Write a Python program to find the average of three students' marks and determine their grades based on the following grading system:

Grade A: 80 or above

Grade B: 60 to 79

Grade C: 40 to 59

Grade D: Below 40

You should use mathematical operators and basic arithmetic operations to achieve this.

Example Input

```
marks_student1 = 85
marks_student2 = 72
marks_student3 = 54
```

Expected Output

Average Marks: 70.33

Grade: B

Solution

Here's how you can solve this problem using only mathematical operators and basic arithmetic:

```
# Marks of the three students
marks_student1 = 85
marks_student2 = 72
marks_student3 = 54

# Calculate the average marks
total_marks = marks_student1 + marks_student2 + marks_student3
average_marks = total_marks / 3

# Determine the grade based on the average marks
grade = (
    (average_marks >= 80) * 'A' +
    (average_marks >= 60) * (average_marks < 80) * 'B' +
    (average_marks >= 40) * (average_marks < 60) * 'C' +
    (average_marks < 40) * 'D'
)

# Print the average marks and grade
print("Average Marks:", round(average_marks, 2))
print("Grade:", grade)
```

Explanation

Calculate the Average: Sum the marks of the three students and divide by 3 to get the average.
Determine the Grade: Use arithmetic operations to conditionally concatenate the grade based on average marks without using if conditions. Each condition contributes to the final grade string.

Question 1: How do you create a string variable for a student's name?

Example Answer:

```
student_name = "John Doe"
```

Question 2: How can you concatenate a student's first and last name into a full name?

Example Answer:

```
first_name = "John"
last_name = "Doe"
full_name = first_name + " " + last_name
# full_name will be "John Doe"
```

Question 3: How do you find the length of a student's name?

Example Answer:

```
student_name = "John Doe"
name_length = len(student_name)
# name_length will be 8
```

Question 4: How can you access the first character of a student's name?

Example Answer:

```
student_name = "John Doe"
first_character = student_name[0]
# first_character will be "J"
```

Question 5: How do you check if a student's name contains a specific substring (e.g., "John")?

Example Answer:

```
student_name = "John Doe"
contains_substring = "John" in student_name
# contains_substring will be True
```

1. Question: Concatenating Student Names with Grades

Given the following lists:

```
names = ["Alice", "Bob", "Charlie", "Diana"]
grades = [85, 92, 78, 90]
```

Create a new list containing strings that pair each student's name with their grade in the format: "Name: Grade".

Answer:

```
student_data = [f'{names[i]}: {grades[i]}' for i in range(len(names))]
```

Output:

```
['Alice: 85', 'Bob: 92', 'Charlie: 78', 'Diana: 90']
```

2. Question: Extracting Names with Uppercase

Given a list of student names, convert all names to uppercase and create a new list with the uppercase names.

```
names = ["Alice", "Bob", "Charlie", "Diana"]
```

Answer:

```
uppercase_names = [name.upper() for name in names]
```

Output:

```
['ALICE', 'BOB', 'CHARLIE', 'DIANA']
```

3. Question: Finding Students with a Specific Grade Range

Given the lists:

```
names = ["Alice", "Bob", "Charlie", "Diana"]
grades = [85, 92, 78, 90]
```

Extract the names of students who scored between 80 and 90 (inclusive).

Answer:

```
students_in_range = [names[i] for i in range(len(grades)) if 80 <= grades[i] <= 90]
```

Output:

```
['Alice', 'Diana']
```

4. Question: Sorting Students Alphabetically

Given a list of student names, sort them alphabetically without using built-in sort methods.

```
names = ["Charlie", "Bob", "Alice", "Diana"]
```

Answer:

```
sorted_names = sorted(names)
```

Output:

```
['Alice', 'Bob', 'Charlie', 'Diana']
```

5. Question: Reversing the Order of Names

Given a list of student names, reverse the order of the names.

```
names = ["Alice", "Bob", "Charlie", "Diana"]
```

Answer:

```
reversed_names = names[::-1]
```

Output:

```
['Diana', 'Charlie', 'Bob', 'Alice']
```

Question 1: How do you get input from the user in Python?

Answer: You can use the `input()` function to get input from the user. For example:

```
user_input = input("Please enter your name: ")
```

```
print("Hello, " + user_input + "!")
```

The `input()` function reads a line of text from the user and returns it as a string.

Question 2: How can you convert user input to an integer?

Answer: You can use the `int()` function to convert a string input to an integer. For example:

```
user_input = input("Enter a number: ")
```

```
number = int(user_input)
```

```
print("The number you entered is", number)
```

It's a good practice to handle exceptions in case the input isn't a valid integer:

```
try:
```

```
    number = int(user_input)
```

```
except ValueError:
```

```
    print("That's not a valid number.")
```

Question 3: How can you convert user input to a float?

Answer: You can use the `float()` function to convert a string input to a floating-point number. For example:

```
user_input = input("Enter a decimal number: ")
```

```
number = float(user_input)
```

```
print("The decimal number you entered is", number)
```

Again, handle exceptions if needed:

```
try:
```

```
    number = float(user_input)
```

```
except ValueError:
```

```
    print("That's not a valid decimal number.")
```

Question 4: How do you handle multiple inputs from the user?

Answer: You can get multiple inputs by calling `input()` multiple times or by splitting a single input string. For example:

```
name = input("Enter your name: ")
```

```
age = input("Enter your age: ")
```

```
print(f"Name: {name}, Age: {age}")
```

Alternatively, if you want to input multiple values at once:

```
user_input = input("Enter your name and age separated by a space: ")
```

```
name, age = user_input.split()
print(f"Name: {name}, Age: {age}")
```

Question 5: How do you ensure that user input is valid (e.g., within a certain range)?

Answer: You can use loops to validate user input. For example, to ensure an age is within a valid range:

```
while True:
    try:
        age = int(input("Enter your age: "))
        if 0 <= age <= 120:
            break
        else:
            print("Age must be between 0 and 120.")
    except ValueError:
        print("That's not a valid number.")
print(f"Your age is {age}.")
```

Certainly! Here's a simple Python script that prompts the user with five questions, collects their responses, and then displays their answers:

```
# Define the questions
questions = [
    "What is your name?",
    "How old are you?",
    "What is your favorite color?",
    "What is your hometown?",
    "What is your favorite hobby?"
]
# Create an empty dictionary to store the answers
answers = {}
# Loop through the questions and get user input
for question in questions:
    # Ask the question and store the answer
    answer = input(question + " ")
    # Store the answer in the dictionary
    answers[question] = answer
# Display the answers
print("\nHere are your answers:")
for question, answer in answers.items():
    print(f"{question} {answer}")
```

How it works:

1. **Questions List:** Contains the questions to ask the user.
2. **Dictionary for Answers:** Stores the answers keyed by the questions.
3. **Loop through Questions:** Uses input() to get user responses and saves them.
4. **Display Answers:** Prints out the collected answers.

1. Simple Input and Output

Question: Ask the user for their name and greet them.

```
name = input("What's your name? ")
print(f"Hello, {name}!")
```

2. Numeric Input and Calculation

Question: Ask the user for two numbers and calculate their sum.

```
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
sum_result = num1 + num2
print(f"The sum of {num1} and {num2} is {sum_result}.")
```

3. String Manipulation

Question: Ask the user for a word and print it in reverse.

```
word = input("Enter a word: ")
reversed_word = word[::-1]
print(f"The reverse of '{word}' is '{reversed_word}'.")
```

4. Concatenating Strings

Question: Ask the user for their first and last names and display the full name.

```
first_name = input("Enter your first name: ")
last_name = input("Enter your last name: ")
full_name = first_name + " " + last_name
print(f"Your full name is {full_name}.")
```

5. Basic Mathematical Operation

Question: Ask the user for a number and calculate its square.

```
number = float(input("Enter a number: "))
square = number * number
print(f"The square of {number} is {square}.")
```

These examples show how to take input from the user and perform various operations without using control structures like loops or conditionals.

Student Data Context

```
students = [
    {"name": "Alice", "age": 20, "grades": [88, 92, 75]},
    {"name": "Bob", "age": 22, "grades": [82, 78, 85]},
    {"name": "Charlie", "age": 23, "grades": [95, 89, 91]}
]
```

Question 1:

Q: How can you create a reference to the first student's grades and modify them? What happens to the original data?

A: Assign the grades of the first student to a new variable. Modifying this new variable will also change the original data since it references the same list object.

Example:

```
python
Copy code
first_student_grades = students[0]["grades"]
first_student_grades[0] = 90
# students[0]["grades"] is now [90, 92, 75]
```

Question 2:

Q: How can you assign values from the students list to three different variables using tuple unpacking?

A: Tuple unpacking allows you to assign the contents of the list to multiple variables.

Example:

python

Copy code

```
student1, student2, student3 = students
```

```
# student1, student2, and student3 now contain the respective student dictionaries.
```

Question 3:

Q: Demonstrate how to extract the ages of all students into a list using list comprehension.

A: List comprehension can be used to create a new list containing the ages of the students.

Example:

python

Copy code

```
ages = [student["age"] for student in students]
```

```
# ages is [20, 22, 23]
```

Question 4:

Q: How can you use a list method to remove the last student from the students list and store the removed student's data?

A: The pop() method removes and returns the last item in a list.

Example:

python

Copy code

```
last_student = students.pop()
```

```
# last_student contains Charlie's data, and students now has only Alice and Bob.
```

Question 5:

Q: Explain how to concatenate two lists of student dictionaries without using loops or functions.

A: Use the + operator to concatenate two lists.

Example:

python

Copy code

```
new_students = [{"name": "David", "age": 24, "grades": [80, 85, 90]}]
```

```
all_students = students + new_students
```

```
# all_students now contains the original students and the new student David.
```

Question 6:

Q: How can you use indexing to access the name of the second student and change it?

A: Use indexing to directly access and modify the name field.

Example:

python

Copy code

```
students[1]["name"] = "Bobby"
```

```
# students[1]["name"] is now "Bobby"
```

Question 7:

Q: Describe how to reverse the order of the students list using a list method.

A: The reverse() method reverses the elements of a list in place.

Example:

python

Copy code

```
students.reverse()
```

```
# The order of students is now reversed.
```

Question 8:

Q: How can you create a shallow copy of the students list without using functions?

A: You can create a shallow copy by slicing the entire list.

Example:

```
python
Copy code
shallow_copy_students = students[:]
# shallow_copy_students is a shallow copy of students.
```

Question 9:

Q: What is the effect of changing a mutable element in a shallow copy of the students list?

A: Changes to mutable elements (like lists) in a shallow copy will affect the original list because they refer to the same objects.

Example:

```
python
Copy code
shallow_copy_students[0]["grades"][0] = 100
# students[0]["grades"][0] is now 100 as well, since both lists reference the same grades list.
```

Question 10:

Q: How can you use the in operator to check if a specific student's name is present in the students list?

A: The in operator can be used to check for membership in a list of dictionaries by checking against a list of names.

Example:

```
python
Copy code
names = [student["name"] for student in students]
is_alice_present = "Alice" in names
# is_alice_present is True if "Alice" is in the list of names, otherwise False.
```

1. Question: Check if a number is in a list.

```
numbers = [1, 2, 3, 4, 5]
num = 3
if num in numbers:
    print(f"{num} is in the list.")
else:
    print(f"{num} is not in the list.")
```

2. Question: Check if all elements in a list are greater than a certain value.

```
numbers = [10, 20, 30, 40]
threshold = 15
if all(x > threshold for x in numbers):
    print("All elements are greater than", threshold)
else:
    print("Not all elements are greater than", threshold)
```

3. Question: Check if any element in a list is even.

```
numbers = [1, 3, 5, 8, 11]
if any(x % 2 == 0 for x in numbers):
    print("There is at least one even number in the list.")
else:
    print("There are no even numbers in the list.")
```

4. **Question:** Determine if the list is empty.

```
numbers = []
if not numbers:
    print("The list is empty.")
else:
    print("The list is not empty.")
```

5. **Question:** Compare two lists to see if they are identical.

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
if list1 == list2:
    print("The lists are identical.")
else:
    print("The lists are not identical.")
```

6. **Question: Determine if a year is a leap year.**

Problem: Write a Python program that checks if a given year is a leap year. A leap year is divisible by 4, but if it's divisible by 100, it should also be divisible by 400.

Answer:

```
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(f"{year} is a leap year.")
else:
    print(f"{year} is not a leap year.")
```

7. **Question:** Find the largest of three numbers.

Problem: Write a Python program to find the largest of three numbers provided by the user.

Answer:

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
num3 = float(input("Enter third number: "))
if num1 >= num2 and num1 >= num3:
    largest = num1
elif num2 >= num1 and num2 >= num3:
    largest = num2
else:
    largest = num3
print("The largest number is:", largest)
```

8. **Question:** Check if a string is a palindrome.

Problem: Write a Python program that checks if a given string is a palindrome. A palindrome is a string that reads the same backward as forward.

Answer:

```
string = input("Enter a string: ")
if string == string[::-1]:
    print("The string is a palindrome.")
else:
    print("The string is not a palindrome.")
```

9. **Question:** Grade evaluation based on marks.

Problem: Write a Python program that assigns grades based on marks entered by the user. The grading scale is as follows:

- Marks >= 90: Grade A
- Marks >= 80: Grade B
- Marks >= 70: Grade C
- Marks >= 60: Grade D
- Marks < 60: Grade F

Answer:

```
marks = int(input("Enter your marks: "))
```

```

if marks >= 90:
    print("Grade A")
elif marks >= 80:
    print("Grade B")
elif marks >= 70:
    print("Grade C")
elif marks >= 60:
    print("Grade D")
else:
    print("Grade F")

```

10. Leap Year Checker

Question: Write a Python program that checks whether a given year is a leap year.

```

year = 2024
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(f"{year} is a leap year.")
else:
    print(f"{year} is not a leap year.")

```

11. Check if a Person is Eligible to Vote

Question: Write a Python program to check if a person is eligible to vote. Assume the voting age is 18.

Answer:

```

age = 17
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")

```

Python program that accepts the cost price of a bike and then calculates and displays the road tax based on the cost:

12. Road Tax Calculation Rules:

- If the cost price is less than 50,000, the road tax is 5%.
- If the cost price is between 50,000 and 1,00,000 (inclusive), the road tax is 10%.
- If the cost price is more than 1,00,000, the road tax is 15%.

Python Program:

```

# Accept the cost price of the bike from the user
cost_price = float(input("Enter the cost price of the bike: "))
# Determine the road tax based on the cost price
if cost_price < 50000:
    tax = 0.05 * cost_price
elif 50000 <= cost_price <= 100000:
    tax = 0.10 * cost_price
else:
    tax = 0.15 * cost_price
# Display the road tax
print(f"The road tax for a bike with a cost price of {cost_price} is {tax:.2f}")

```

Example Output:

Enter the cost price of the bike: 75000

The road tax for a bike with a cost price of 75000.0 is 7500.00

13. Python program to calculate the electric bill based on the given criteria:

Criteria:

1. The first 100 units are free.
2. The next 200 units (i.e., units between 101 and 300) are charged at 2 Rs per unit.
3. Units above 300 are charged at 5 Rs per unit.

Python Program:

Accept the total units consumed from the user

```
units = float(input("Enter the total units of electricity consumed: "))
# Initialize the bill amount
bill_amount = 0
# Calculate the bill based on the given criteria
if units > 300:
    bill_amount += (units - 300) * 5
    units = 300
if units > 100:
    bill_amount += (units - 100) * 2
# Display the calculated bill amount
print(f"The total electric bill is: {bill_amount:.2f} Rs")
```

Example Output:

Enter the total units of electricity consumed: 350

The total electric bill is: 400.00 Rs

Explanation:

1. **Input:** The program asks the user for the total number of units consumed.
2. **Calculation:**
 - If units exceed 300, it calculates the cost for units above 300 at 5 Rs per unit.
 - It then calculates the cost for units between 101 and 300 at 2 Rs per unit.
 - Units up to 100 are free, so they don't affect the bill.
3. **Output:** The program prints the total electric bill formatted to two decimal places.

14.